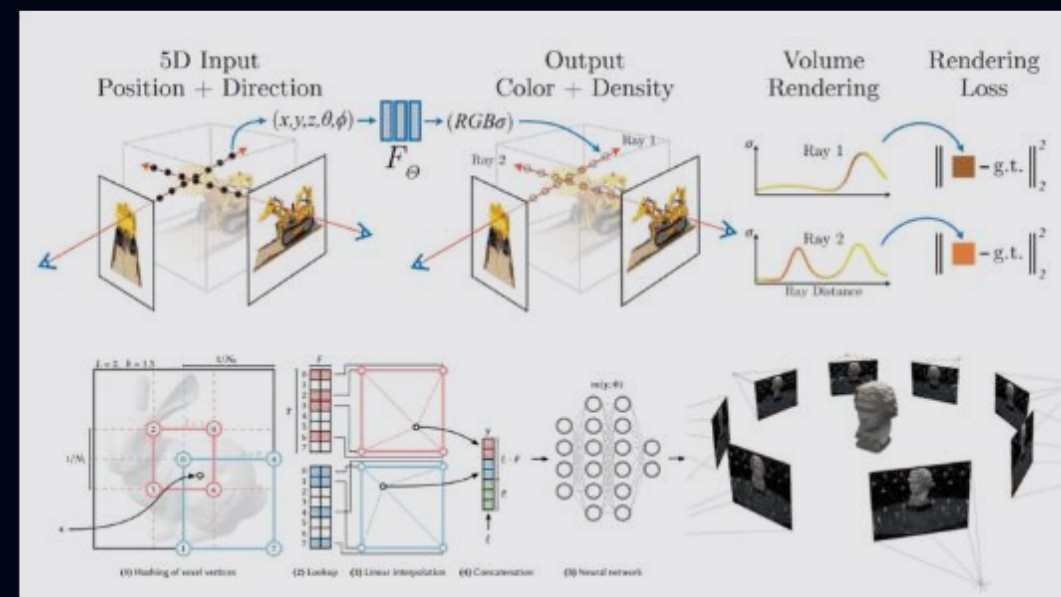


Interim Progress: **Adversarial Geometric Distillation**

Mitigating Hallucinations and the Janus Problem in 3D Model Generation

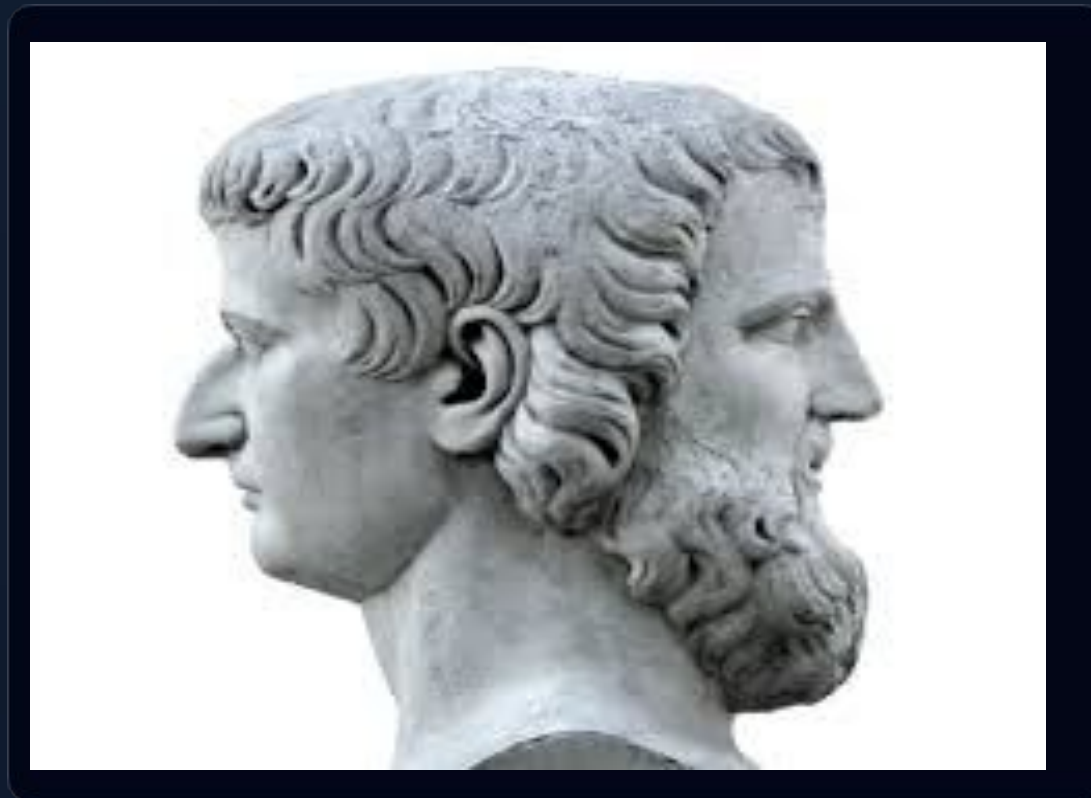
Background: Text-to-3D Synthesis

- The field evolved from optimization-based reconstruction to zero-shot synthesis utilizing **Neural Radiance Fields (NeRF)** and **Score Distillation Sampling (SDS)**.
- SDS utilizes a pretrained 2D diffusion model to provide gradients for a 3D model by minimizing noise differences.
- While enabling high-fidelity synthesis, these pipelines operate purely in 2D pixel space, missing crucial 3D structural awareness.



The Janus Problem & Hallucinations

- **Mode Collapse:** 2D diffusion models are biased toward frontal views. SDS optimization often settles into these high-probability 2D viewpoints.
- This creates the **Janus Problem:** generating multi-faced artifacts or redundant limbs on a single object.
- Other topological errors include floating components (Betti number > 1), unintended handles (genus > 0), and disconnected geometry.



Literature Review: SOTA Limitations



Magic3D / Fantasia3D

Focuses on high-res geometry and disentangling appearance. However, the generative priors remain fundamentally "geometry blind".



Perp-Neg & Entropic SDS

Attempt mathematical debiasing in score-space, but fail to explicitly understand if the output structure is anatomically correct.



Hallo3D

Introduces multimodal detection. Converts spatial errors into textual feedback ("negative prompts"), losing critical spatial precision.

Defining the Research Gap

1. Geometry Blindness

The optimization objective for 3D generation lacks a geometric energy term defined directly on the mesh's topological and spectral properties. Gradients reshape 2D pixels, not 3D manifolds.

2. The Semantic Bottleneck

Frameworks like Hallo3D map spatial hallucinations into natural language. This many-to-one lossy compression discards exact vertex coordinates, leading to over-correction or new artifacts.

Research Objectives



1. Literature Review Analyze the evolution of geometric hallucinations and how specific topological errors manifest in modern generative 3D models.



2. Method Study Examine adversarial neural networks and self-distillation technologies used for automated artifact detection and mesh refinement.



3. System Design Develop an Adversarial Geometric Distillation (AGD) framework that integrates an auxiliary AI critic to correct 3D anomalies without text prompts.



4. Evaluation Assess performance using quantitative geometric metrics (surface smoothness, manifold quality) against baseline SOTA models.

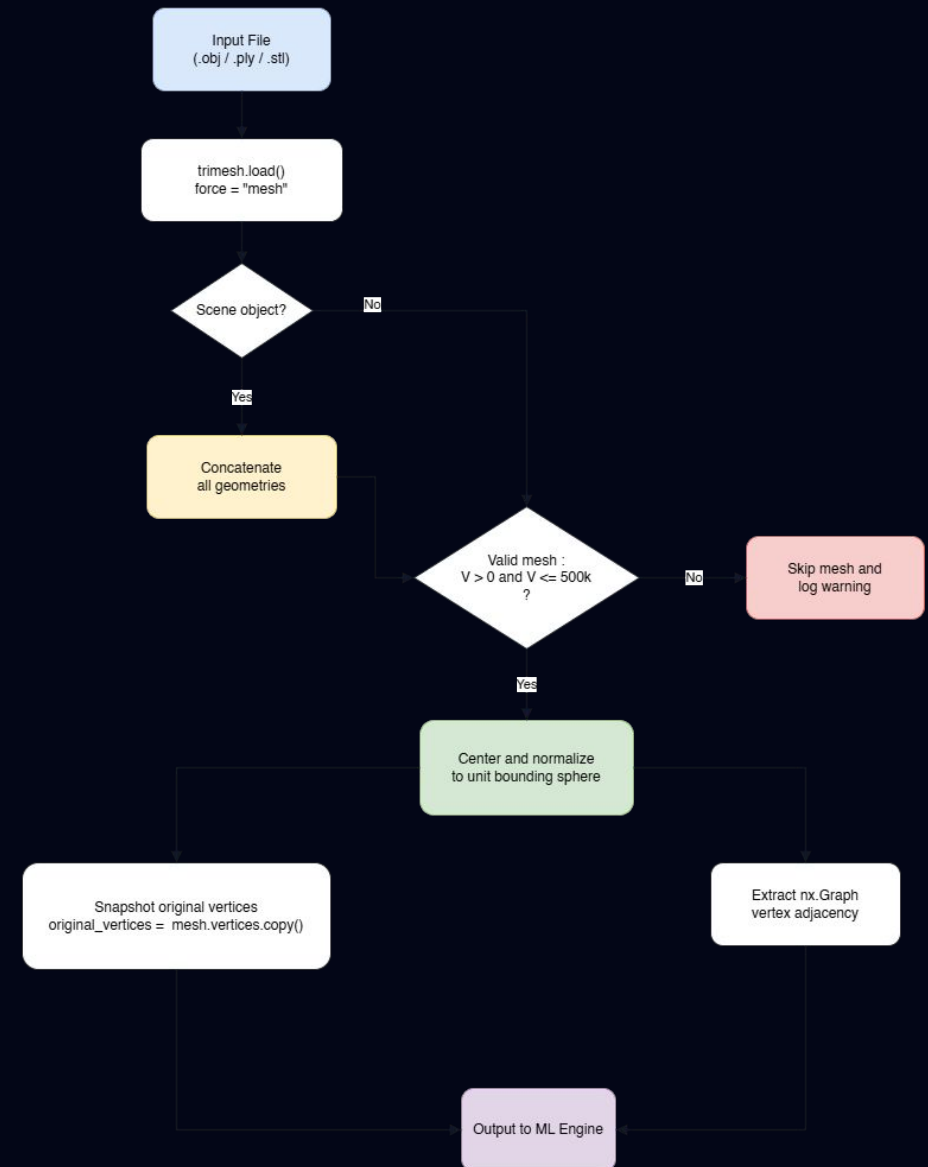
The AGD Hypothesis

Replacing imprecise textual feedback with direct geometric grounding will eliminate structural inconsistencies and ensure exact topological integrity in zero-shot synthesis.

System Architecture

- **1. Preprocessing Module:** Normalizes meshes and extracts adjacency graphs.
- **2. ML Engine:** Fuses signals from a topological Discriminator, a GNN-based Critic, and an LMM multi-view inspector (LLaVA-v1.6).
- **3. Extended Module:** Projects numerical severity scores into explicit vertex weights, driving the grounded refinement optimization.

Diagram 5.2 - Preprocessing Module Data Flow



Pipeline: Ingestion & Rendering

Stage 1: Pre-processing

Loads raw meshes via *Trimesh*. Meshes are centered and normalized to a unit bounding sphere. Vertex normals are computed, and the adjacency graph is extracted as a sparse matrix for downstream spectral analysis.

Stage 2: Dense Multi-View Rendering

A pure-NumPy software rasterizer captures N views (e.g., 36) using Fibonacci-sphere distribution. Each viewpoint renders 5 specific buffers: RGB, Depth, Normal Map, Silhouette, and Edge/Curvature map.

Pipeline: Detection & Critic

Stage 3: LMM Detection

LLaVA v1.6 inspects RGB buffers, outputting a numerical severity score [0,1] for spatial errors. Critically, we discard natural language notes and strictly ingest the numerical bias to avoid semantic loss.

Stage 4: Adversarial Geometric Critic

Computes global scores analyzing Laplacian eigenvalue variance. We extract exact algebraic topological invariants like the Euler Characteristic:

$$\chi = V - E + F$$

This allows absolute mathematical validation of handles and loops.

Pipeline: Geometric Grounding

A large, stylized blue '3D' text graphic, where the '3' and 'D' are connected and have a slight 3D effect with a shadow.

SPATIAL PRECISION

Mapping Views to Vertices

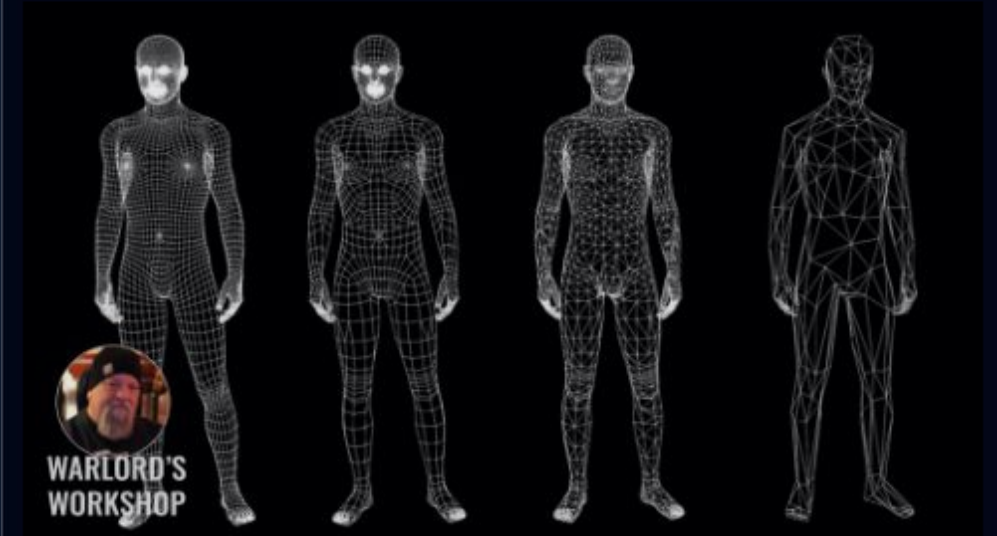
The grounding module solves the core research problem. It projects 2D severity scores back onto specific vertices by calculating projection distance and normal vector alignment. This fuses LMM bias with the GNN critic anomaly scores into a unified, high-precision vertex weight map.

Pipeline: Grounded Refinement Loop

Iteratively updates vertex positions using an energy minimization loop guided by the grounded weight map.

$$E(v_i) = \lambda \nabla L_{\text{geo}}(v_i) + (1 - \lambda) \nabla D(v_i)$$

The distillation anchor term $D(\mathbf{v})$ guarantees theoretical boundedness, ensuring the mesh doesn't drift into degenerate geometry during Laplacian smoothing.



Technology Stack



Geometry & Compute

Python forms the base, utilizing **Trimesh** for mesh loading/topology and **NumPy/SciPy** for custom multi-view software rasterization and sparse Laplacian eigenvalues.



Machine Learning

PyTorch and **PyTorch Geometric** construct the GCN Critic. Allows for dynamic, gradient-based anomaly scoring directly on graph edge indices.

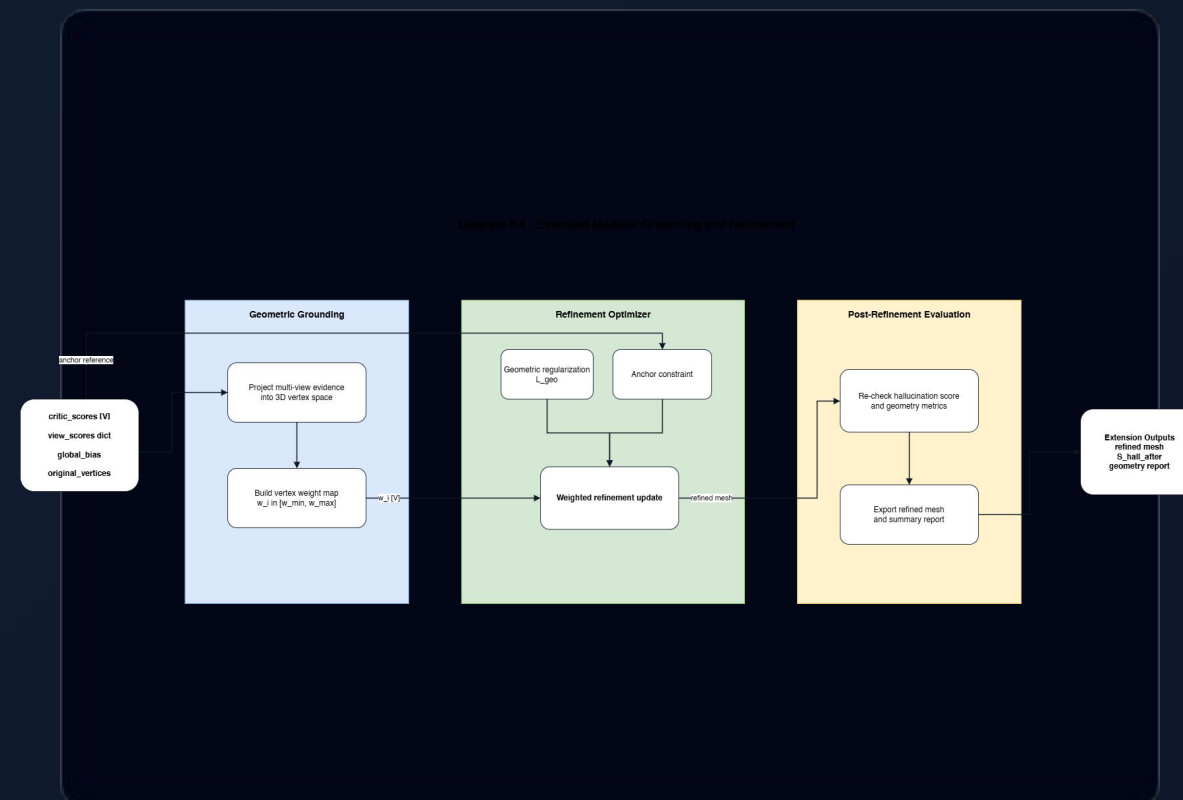


Vision Language

Local deployment of **LLaVA-v1.6** (via HuggingFace) executes hardware-accelerated multimodal inspection across 36+ generated viewpoints.

Experiments Conducted

- Iterative mathematical validation was performed on a highly curated dataset consisting of 21 test meshes.
- Evaluated against Objaverse baseline outputs. Successfully corrected explicitly hallucinated models (Janus faces, duplicated limbs).
- Developed continuous integration "smoke tests" bypassing the heavy LMM module to guarantee the mathematical stability of the Laplacian solvers during code refactoring.



Progress & Timeline



Questions?

Thank you for your time.
